1. Practical 6

Develop a program that executes the equivalent of the shell command: ls -l | grep ".c" using fork(), pipe(), dup2(), and exec() system calls.

meghana@hp1: ~

GNU nano 8.7.1 practical6b.c

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];

    pipe(fd);

    if (fork() == 0)
    {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);
    }

    if (fork() == 0)
    {
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        close(fd[1]);

        execlp("grep", "grep", ".c", NULL);
    }

    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

    return 0;
}
```

```
meghana@hp1:~$           $ nano practical6b.c
meghana@hp1:~$           $ gcc practical6b.c -o practical6b
meghana@hp1:~$           $ ./practical6b
-rwxr-xr-x 1 sri_dhruthi sri_dhruthi 16000 Aug 16 10:54 ExecExample
-rw-r--r-- 1 sri_dhruthi sri_dhruthi   181 Aug 16 11:00 ExecExample.c
-rwxr-xr-x 1 sri_dhruthi sri_dhruthi 16056 Aug 16 10:52 MultipleChildProcess
-rw-r--r-- 1 sri_dhruthi sri_dhruthi   238 Aug 16 11:00 MultipleChildProcess
.c
-rwxr-xr-x 1 sri_dhruthi sri_dhruthi 16056 Aug 14 10:08 Multiplechildprocess
-rw-r--r-- 1 sri_dhruthi sri_dhruthi   258 Aug 14 10:06 Multiplechildprocess
.c
-rw-r--r-- 1 sri_dhruthi sri_dhruthi   297 Aug 16 10:58 ParentChild.c
-rw-r--r-- 1 sri_dhruthi sri_dhruthi   336 Aug 16 10:59 ParentWait.c
-rwxr-xr-x 1 sri_dhruthi sri_dhruthi 16000 Aug 16 10:53 ProcessID
-rw-r--r-- 1 sri_dhruthi sri_dhruthi   114 Aug 16 10:57 ProcessID.c
-rw-r--r-- 1 sri_dhruthi sri_dhruthi   578 Sep 18 03:44 addresstranslating.c
-rw-r--r-- 1 sri_dhruthi sri_dhruthi    79 Sep 28 15:24 combined.txt
```

2. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    if(sig==SIGINT)
        printf("\nSIGINT Received\n");

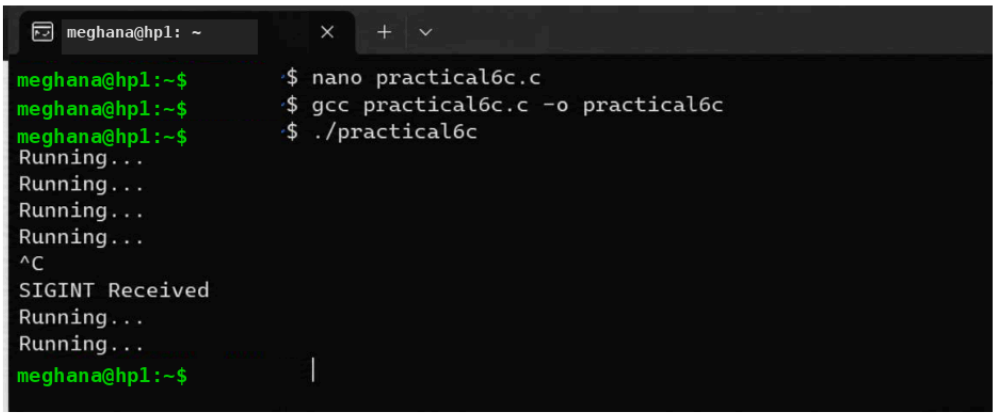
    if(sig==SIGTERM)
        printf("\nSIGTERM Received\n");

    if(sig==SIGUSR1)
        printf("\nSIGUSR1 Received\n");
}

int main()
{
    signal(SIGINT,handler);
    signal(SIGTERM,handler);
    signal(SIGUSR1,handler);

    while(1)
    {
        printf("Running...\n");
        sleep(3);
    }

    return 0;
}
```



Ex; kill -SIGUSR1 PID
Ex: kill -SIGTERM PID